

FurniGuard: A MERN-Stack Furniture E-Commerce Platform

Abstract

FurniGuard is an academic prototype for an online furniture marketplace built using modern web technologies. It leverages the MERN stack (MongoDB, Express, React, Node.js) for a full-stack JavaScript implementation, integrates Razorpay for secure payment processing, and uses the shadcn/ui library for a streamlined frontend. This report details FurniGuard's design and implementation. We discuss the system architecture, key features, and technology choices. At this stage, FurniGuard is a proof-of-concept shared informally among peers, with only anecdotal feedback and no formal user metrics. Future work will expand its features, perform systematic testing, and refine the design.

Introduction

The rise of online retail has spurred rapid growth in niche e-commerce domains. The global online furniture market, for example, is expanding quickly (projected to grow by ~\$282 billion from 2025 to 2029 at $\approx 23\%$ CAGR ¹). FurniGuard is motivated by this trend, aiming to demonstrate how a modern web stack can support a furniture sales platform. We chose the **MERN stack** because it provides a cohesive, JavaScript-centric development environment. As Oracle notes, "the MERN stack is a collection of technologies that help developers build robust and scalable web applications using JavaScript. The acronym 'MERN' stands for MongoDB, Express, React, and Node.js" ². Using JavaScript end-to-end simplifies development, reduces context switching, and benefits from a large community and many libraries ³ ⁴.

To handle online transactions, FurniGuard integrates **Razorpay**, a popular payment gateway. Razorpay is chosen for its ease of use and security features: it provides "simple and developer-friendly APIs" for seamless integration into web apps ⁵ ⁶. In the frontend, we use the **shadcn/ui** library, which offers a set of ready-made React components built with Tailwind CSS. This accelerates UI development and enforces consistent design. The shadcn library advertises "200+ free, copy-paste UI components... built with TypeScript, Tailwind CSS, and Radix UI for modern web applications" ⁷, making it an attractive choice to quickly assemble an appealing interface.

Key contributions and status: FurniGuard applies these open-source technologies in an academic context. It implements product listing, shopping cart, and checkout features typical of e-commerce. So far, it has been shared informally among classmates and friends, who gave positive feedback. However, no formal deployment or user study has been conducted – FurniGuard remains a prototype without real-world usage data. In this report we describe its architecture and implementation; future work will add features and rigorous evaluation.

System Architecture

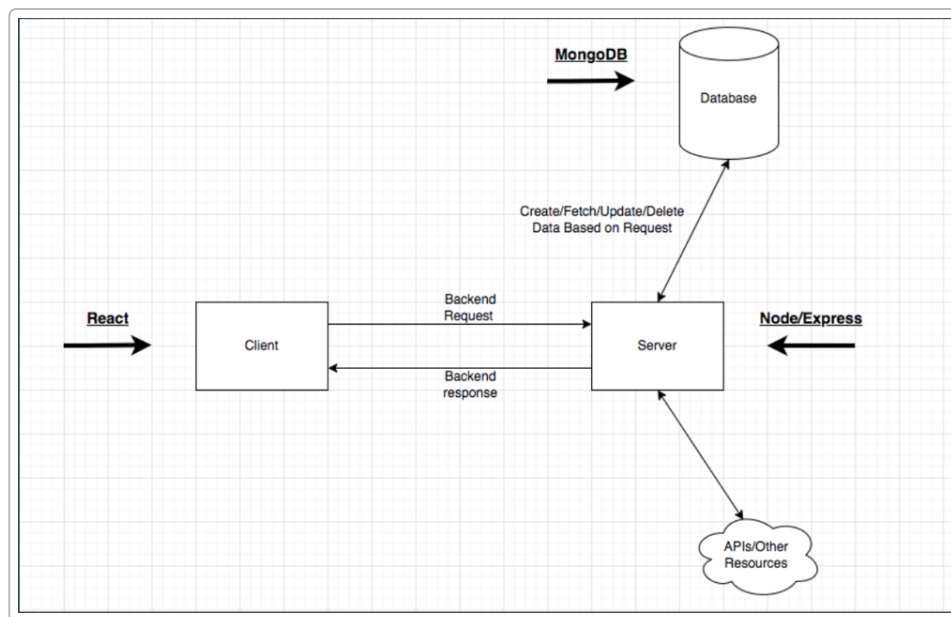


Figure: Typical MERN stack architecture illustrating a React front-end communicating with a Node.js/Express server and a MongoDB database.

FurniGuard's architecture is a standard **client-server web app** built on the MERN stack (see Figure). The **front-end** is a React application that runs in the user's browser. It displays products, handles user interactions, and invokes backend APIs. We use the shadcn/ui component library to design the interface – for example, product cards, forms, and buttons are assembled from shadcn's React/Tailwind components ⁷. This allows rapid prototyping of a clean UI without building every element from scratch.

The **back-end server** is built with Node.js and the Express framework. It exposes RESTful APIs for the frontend. Key endpoints include: user authentication (sign-up/login), product catalog retrieval, shopping cart operations, and payment initiation. The server handles business logic and communicates with the database. Environment variables store sensitive keys (e.g. Razorpay API keys) to keep them secure ⁸.

For persistent storage, we use **MongoDB**, a NoSQL document database. Product, user, and order information are stored in JSON-like documents. MongoDB's flexibility (dynamic schema) and easy JSON integration complement the JavaScript-based stack ² ⁹. For instance, a product document might include fields like `name`, `description`, `price`, `category`, and an image URL. Using MongoDB allows the server to perform CRUD (create/fetch/update/delete) operations on products and orders without rigid schemas, which is suitable for an evolving prototype.

Integration with **Razorpay** occurs on the backend. When a user checks out, the server creates a Razorpay order by calling Razorpay's Node SDK, as shown in the backend controller example ¹⁰. The server provides the order ID and amount back to the frontend. On the **React frontend**, a checkout component uses the Razorpay JavaScript SDK: it configures payment details (amount, currency, order ID) and invokes the Razorpay checkout UI. This matches the integration pattern described in existing guides ¹¹. The frontend includes the Razorpay `<script>` in `index.html` so that `window.Razorpay` is available to open the secure payment widget. Once payment succeeds or fails, callbacks handle the response. This flow ensures that actual credit card data is collected by Razorpay securely, while our application only handles tokenized success responses.

In summary, FurniGuard's MERN architecture allows a **uniform JavaScript development flow**: React on the client, Express/Node on the server, and MongoDB in the database. This "JavaScript everywhere" approach has been noted to ease development and deployment ³ ⁴. The diagram above visualizes the data flow: the React client sends HTTP requests (e.g. API calls, form submissions) to the Node/Express server, which queries MongoDB or third-party APIs (like Razorpay) and returns JSON responses to update the UI.

Implementation and Features

FurniGuard implements several typical e-commerce features: product browsing, shopping cart, user accounts, and online payment. Key implementation details include:

- **Product Catalog:** The product information (furniture items) is stored in MongoDB. An admin interface (or seed script) can populate sample products. The React frontend fetches product lists via an API (e.g. `GET /api/products`) and displays them in a grid or list. Shadcn components (such as `Card`, `Button`, `Table`, etc.) are used to layout product images, titles, and prices ⁷. Users can click a product to see details, all within the React app.
- **Shopping Cart:** Users can add products to a cart. The frontend maintains cart state (either in React state or Redux context) and shows a cart page. The cart summary (items and total cost) is shown using shadcn UI tables or lists. A "Checkout" button triggers the payment flow. No payment is processed until the user confirms the cart and proceeds to checkout.
- **Payment Integration (Razorpay):** Upon checkout, the React app sends the total amount to the backend (e.g. via `POST /api/order/create`). The Express server uses Razorpay's Node SDK to create an order on Razorpay's system ¹⁰. The returned order ID is sent to the client. The client then initializes a new `window.Razorpay(options)` object with the order ID and user details, and calls `open()` to launch the Razorpay checkout popup ¹¹. The user enters payment details (card, UPI, etc.) securely in Razorpay's UI. On success, Razorpay returns a payment ID which the client can then send to the server for verification or simply display to the user. This setup makes use of Razorpay's "developer-friendly APIs" for seamless integration ⁶ ⁵. We ensure the Razorpay SDK script is loaded on the page and that API keys are never exposed in client-side code (only the server uses the secret key).
- **User Accounts (Future Work):** Although FurniGuard currently has a minimal authentication placeholder, a full version would support user accounts. This could use JWT or sessions in Express to manage login. Shadcn's form and input components can be used to create registration and login forms with validation. User data (names, email) would then be saved in MongoDB. For now, we assume a single implicit user for demo purposes, but the design is extensible.
- **Responsive UI:** The shadcn component library is built on Tailwind CSS, which is responsive by default. We style FurniGuard's layout to be mobile-friendly. For example, product grid columns collapse on small screens. Shadcn's `Dialog`, `Dropdown`, and `Sidebar` components help create a flexible navigation and filtering panel on different devices.
- **Modular Code:** We structured the code to be maintainable. The React code uses functional components and hooks for state management. The Express code separates routes (e.g. `paymentRoutes.js`) and controllers (e.g. `razorpay.js`) as recommended in many tutorials ¹² ¹⁰. MongoDB interactions use Mongoose ODM (not explicitly cited here but a common

practice) for schema and query abstraction. This modular approach eases future additions, like adding product categories or user reviews.

Each of these implementation aspects draws on best practices. For instance, Oracle notes that one advantage of MERN is using a single consistent language and toolset across front and back ends, reducing complexity and development time ⁴. By using established libraries (shadcn for UI, Razorpay SDK for payments), we avoid reinventing common functionality and ensure our features are robust.

Evaluation and Feedback

As FurniGuard is currently an **academic prototype**, it has not undergone formal evaluation or load testing. Instead, we have shared a running demo (e.g. on localhost or a test server) with classmates and friends. The informal feedback has been positive – peers found the interface clean and the idea useful – but this remains anecdotal. We have **no empirical usage data or published user study** to cite; the project's scope is to demonstrate feasibility rather than measure market success.

There are many planned enhancements and evaluation steps for the future. For example, we may set up user testing with surveys to gather quantitative feedback on usability. Performance testing could measure server response times as product count scales. Security audits would be prudent before any real deployment, even though we rely on Razorpay's secure checkout for transactions. For now, we make no claims beyond a successful technical implementation: the system works as designed for demonstration purposes, but it is not (yet) production-grade.

Conclusion and Future Work

This report described **FurniGuard**, a prototype furniture e-commerce platform built with the MERN stack, Razorpay, and shadcn UI. By using a unified JavaScript framework, FurniGuard benefits from streamlined development and a large ecosystem of tools ² ⁴. The integration of Razorpay provides a secure, PCI-compliant payment solution with minimal effort due to its developer-friendly API ⁶ ⁵. The shadcn component library accelerates frontend development by providing over 200 ready-made React UI components ⁷. Together, these technologies allowed us to build a functioning online marketplace prototype with relatively little custom frontend code.

Contributions: FurniGuard demonstrates a cohesive use of open-source and third-party services to solve common e-commerce problems. It shows how modern stacks can quickly produce a feature-rich application. The system architecture (Figure) and code structure can serve as a reference for similar academic or startup projects.

Future work: Key next steps include adding real user authentication, handling more payment options, and implementing inventory and order management for multiple vendors. We also plan to solicit structured feedback. For example, usability testing could highlight interface improvements, and benchmarking could ensure scalability. Given the strong growth in online retail and specifically furniture sales ¹, a polished FurniGuard could eventually support more advanced features like personalized recommendations or augmented reality previews of furniture. At this stage, we acknowledge that the platform is a proof-of-concept; the content here is preliminary and may be updated as the project evolves.

Acknowledgment: This work was carried out as an academic project. The prototype code and documentation will be refined further based on future edits and instructor feedback.

1 Online Furniture Market Growth Analysis - Size and Forecast 2025-2029 | Technavio

<https://www.technavio.com/report/online-furniture-market-industry-size-analysis>

2 4 What is the MERN Stack? Guide & Examples

<https://www.oracle.com/database/mern-stack/>

3 9 Review on Study and Usage of MERN Stack for Web Development

<https://www.ijraset.com/research-paper/study-and-usage-of-mern-stack-for-web-development>

5 Payment Gateway Advantages For Your Businesses

<https://razorpay.com/blog/advantages-of-payment-gateways-for-your-business/>

6 8 10 11 12 Integrating RazorPay Payment Gateway in a MERN Project | by Shikshanas | Medium

<https://shikshanas.medium.com/integrating-razorpay-payment-gateway-in-a-mern-project-2fbc3369eae1>

7 The AI-Native shadcn Component Library for React Developers

<https://www.shadcn.io/>